

FDE Analysis — Aurora Voice Agent

Objective 2.4. Synthesizes `SETUP-AND-REVIEW.md`, `EVAL_REPORT.md`, `stage2-evidence/`, and `DEVIATIONS.md` into one judgment: what would actually break, and what has to change first.

1. Framing

The question an FDE is being asked here is not "does the demo work" — it does, reliably, for the scripted path. The question is **what breaks at 3am with a real caller who doesn't follow the script**, and whether the system fails toward a human or fails toward silence when it does. Two concrete answers surfaced during this review, both from live testing, neither from reading the code: a caller asking for a human has a **measured ~40% chance** of actually getting disconnected instead (§2), and the system has **zero retry or fallback** around any live model call — a single malformed tool argument or a single rate-limit response takes the whole call down with a raw traceback, not a graceful handoff (§3). Neither of those is visible from "12/14 evals passed" or "the smoke test works." Both were only visible from manually talking to it and reading the actual `agent.py/voice_loop.py` control flow. That gap — between what a deterministic test suite proves and what a live caller actually experiences — is the whole subject of this document.

2. Model boundaries — what the model decides vs. what code forces

Model decides: caller intent, phrasing of every reply, which tool to call for a given utterance (except where forced — see below), and slot values extracted from free text (dates, guest counts, names, room preferences).

Code forces, concretely, with the exact mechanism:

- `required_tool_for()` (`agent.py`) — pre-empts the model's own tool choice for high-confidence policy/amenity phrases, forcing `search_hotel_knowledge` via `tool_choice` before the first model call. This is the one place the code overrides the model's judgment outright rather than just constraining its inputs/outputs.
- `explicit_language_request()` + `AgentRouter.set_language()` (`agent.py/router.py`) — the model can *propose* a language change by calling the tool, but the change is only accepted if the caller's own utterance names the target language; the model's tool call alone is not sufficient (`RouterTests.test_overeager_language_tool_cannot_change_state` in `test_features.py` directly tests this).
- Structured tool args — every tool call is OpenAI-style JSON function-calling, never free text parsed downstream, and `run_tool()` never trusts a value's type without coercion (`_coerce_guests()`).
- System-prompt scope — the guardrail ("hotel booking support only") is prompt-enforced, not code-enforced; there is no code-level filter that would catch a guardrail bypass the prompt

missed.

Where the boundary is currently too loose — three real, evidenced cases, not hypotheticals:

1. **Compound control actions.** `agent.py`'s tool loop tracks `action` as "the last control signal seen," with zero validation that two control tools shouldn't fire in the same turn. Live testing found the model chaining `transfer_to_human()` immediately followed by `end_call()` in one turn, **2 of 5 times** for the same input (`stage2-evidence/README.md` Finding 1). The caller hears "I'm transferring you" and the system actually hangs up. This is the single most important finding in this entire review — it inverts the one escape hatch (§3) that's supposed to catch everything else.
2. **Guest-count type.** Fixed during this review, but instructive: the model reliably emitted `guests` as a quoted string against an `integer`-typed schema, and Groq's strict validation rejected the call outright, with no code path to recover. The boundary between "what the schema declares" and "what the model actually emits" was simply wrong, in production, for the entire time this bug existed (`DEVIATIONS.md` §1).
3. **Confirmations are entirely un-forced.** Nothing in code checks that a caller's "book it" was a genuine, unambiguous confirmation before `create_booking` fires — it's model judgment end-to-end. Live testing showed the model handling one ambiguous case well (asking which of 5 rooms, `stage2-evidence` Finding 2) but that's model behavior on this run, not a code guarantee; nothing prevents a future prompt change or model swap from silently guessing instead.

Failure taxonomy observed in this review (not a hypothetical list — every entry below actually happened):

- **Wrong/compound tool call:** the `transfer`→`hangup` chaining above.
- **Malformed structured output:** the `guests`-string crash.
- **Silent local fallback:** `say -v <voice-name>` failing silently and using the wrong voice instead of erroring (`DEVIATIONS.md` §3) — not a model failure, but the same *shape* of failure (a boundary that fails quietly instead of loudly), worth grouping with the above because it reveals a house style: this codebase generally does not fail loudly.
- **Hallucinated policy:** not observed in this review — `required_tool_for()` and `grounding.fabricated_policy` both held under adversarial testing. Worth stating explicitly since it's the one boundary that *did* hold every time it was tested.

3. Operational fallbacks

What exists today:

- `PROVIDER=mock` — a full offline fallback for rehearsal/CI, not a production runtime fallback.
- `TTS_BACKEND=system` — a cost fallback, not a reliability one.
- `transfer_to_human` — the intended universal human-escalation escape hatch.
- The guardrail redirect — keeps off-topic conversation from derailing the call.

What's missing, in order of how much it matters given what this review actually found:

- 1. **Any retry/catch around a provider call, anywhere in the CLI path.** `voice_loop.py` has zero `try/except` around `agent.respond()` or `provider.chat()`. A malformed tool call, a rate limit (this review hit Groq's 100,000 TPD limit mid-session and watched retries make the reset time worse, not better — repeated 429s appeared to consume more budget rather than free it up), or a transient network error all produce the same outcome: the entire process dies with a raw Python traceback, and the call ends with **no SIP-mappable signal at all** — not a hangup, not a transfer, nothing. Contrast this directly with `livekit/talk_server.py`, which wraps every handler in `try/except` and degrades to a JSON error without killing the server — the same codebase has two different reliability postures for the same underlying risk, depending only on which front end happens to be running.
- 2. **The transfer/hangup escape hatch can itself silently fail** (§2, Finding 1). "Every failure should end in a human, never a dead line" is the right principle, and this codebase violates it in the one place it matters most: the caller literally asked for a human, and the code's own handling of that request has a measured chance of ending the call instead.
- 3. **No STT low-confidence path.** `voice_loop.py`'s real-mic capture (§4 latency data) has no confidence signal threaded through at all — a garbled or low-confidence transcript is handed to the LLM exactly like a clean one.
- 4. **No no-input/no-match reprompt ladder.** Classic IVR design reprompts once, then twice differently, then escalates — nothing here distinguishes "caller said nothing" from "caller said something off-script" beyond the guardrail's fixed refusal text.
- 5. **No degraded-mode script.** If the LLM is down, there's no scripted fallback ("I'm having trouble right now, let me get you the front desk") — the alternative to a working model call is a crash, not a degraded experience.

4. Latency

Per-stage budget, as actually measured (not estimated — real numbers, both from live-OpenAI text mode and, uniquely, from a real-microphone session; full data in `stage2-evidence/`):

Stage	Text-mode range (12-turn session)	Real-mic session (3 turns)
capture	n/a (no mic in text mode)	8955-11238ms — mostly the caller talking , not system time
stt	n/a	965-1678ms
routing	~0ms	~0ms
retrieval	0-1ms	n/a (no RAG turn in this sample)

llm	663-3654ms	1122-2050ms
tools	0-1ms	0-0.1ms
tts (spoken, system backend)	5074-23948ms	2750-10370ms
total	6809-27604ms	14859-24412ms

Where the turn-time actually goes: overwhelmingly TTS, and this needs a real caveat, not a naive read. `TTS_BACKEND=system` calls `subprocess.run(["say", ...])`, which **blocks until the entire reply has finished being spoken** — `say` starts making sound almost immediately, but the code has no way to observe or log that moment, only when the whole utterance is done. So a `tts: 10370ms` reading means "the full reply took 10.4 seconds to finish," not "the caller waited 10.4 seconds to hear anything." **The system does not currently measure first-audio latency at all**, because the underlying TTS call is fully synchronous — there is no field in `telemetry.py`'s schema for it, and there could not be one without an architecture change to streaming TTS.

The caller's actual perceived latency — the honest number — is closer to `capture + stt + routing + llm`: on the real-mic session's first turn, that's $11238 + 1678 + 0.1 + 1122 \approx \mathbf{14 \text{ seconds}}$ before Aurora's voice starts at all. That is the number that should be compared against a production target, not the `total` field, and not the `tts` field alone.

Endpointing silence threshold (`ENDPOINT_SILENCE_MS`, default 600ms) is the one latency-facing tunable actually exposed. It's a direct responsiveness-vs-cutoff tradeoff: lower it and a caller with a mid-sentence pause gets cut off early (their turn ends before they meant it to); raise it and every turn gains flat dead air equal to the difference, on every single turn, for every caller, whether or not they paused. This review did not run the `ENDPOINT_SILENCE_MS=350` vs `900` side-by-side comparison `RUNBOOK.md` describes (that needs interactive mic testing across multiple runs) — worth doing before tuning this for a real deployment.

Perceived vs. actual latency: this system has no filler audio, no "let me check that for you," and no streaming TTS — the caller experiences the full `capture+stt+llm` gap as silence, then the full reply plays back in one blocking chunk. A production system would mask the first ~1-2 seconds with an acknowledgment sound or phrase and stream TTS so the caller hears the first words within a few hundred milliseconds of the model starting to generate — neither exists here.

Targets to hold (proposed, since none exist in the repo today): first-audio-out p95 under ~1.5s (this system, measured: ~14s on the one real-mic sample — roughly an order of magnitude over); full-turn p95 (caller-perceived, meaning until the reply is fully delivered) under ~4-6s for a simple lookup, ~8-10s for a grounded/tool-calling turn. Both targets are illustrative, not industry-cited — the point is that this system has no targets at all today, and the gap to any

reasonable target is large and measured, not assumed.

5. Observability

What `telemetry.py` emits today: `TurnTrace` produces one JSONL line per turn with a `traceId/sessionId/turnId`, per-stage `timings` (whichever stages actually ran that turn — note turn 1 of the real-mic session has no `tools` key at all, because no tool fired, which is correct but easy to misread as a missing field rather than "zero tool calls this turn"), an ordered `events` list, and `attributes` (language, locale, provider, model, action, sources). `_sanitize()` redacts by **key name** (`_SENSITIVE_KEYS`: `contact`, `email`, `guest_name`, `name`, `phone`; `_CONTENT_KEYS`: `message`, `query`, `result`, `text`, `transcript`, omitted-by-default via `TELEMETRY_INCLUDE_CONTENT`) recursively through nested structures — a genuinely solid, low-effort redaction design, with one real gap: it's a fixed keyword list, so a future tool argument holding sensitive data under an unlisted key (e.g. a `notes` field with a phone number in free text) would not be caught.

What's missing for production:

- **Per-stage spans exist but aren't aggregated anywhere** — every turn's JSONL line is independent; there's no rollup, no dashboard, no percentile computation over the raw data this review had to compute by hand (§4's tables).
- **Tool success/failure rates** — there is no failure event type distinct from a successful `tool.result` for cases like the guests-string crash; that failure mode currently produces no telemetry event at all, because the process dies before `trace.finish()` is ever called.
- **Containment vs. transfer rate** — the single most important call-center KPI for this system (did the agent resolve it, or did it need a human) isn't computed or even directly derivable without replaying every JSONL line and checking the terminal `action`.
- **Barge-in counts** — `livekit/talk_server.py` has barge-in-adjacent logic (`_is_probable_playback_echo`) but no counter or rate is emitted; the CLI paths have no barge-in concept at all.
- **Alerting thresholds** — none exist anywhere in this codebase; telemetry is written, never read back or acted on.

The call-center KPIs this should roll up to: containment rate (resolved without a human), transfer rate and *transfer success rate* (given §2's Finding 1, "transfer was requested" and "transfer actually happened" are now known to be different numbers here), average handle time, first-contact resolution, abandonment rate (caller hangs up before resolution), and cost per contained call. None of these exist today even in principle from the current telemetry without manual analysis.

6. Scale & cost

`scale_check.py --dau 10000000` (defaults): **250,000 calls/day, 694.4 average concurrent, 5,555.6 peak concurrent, 7,223 provisioned sessions, 181 workers**, \$0.00/day cost (cost defaults to zero and must be explicitly supplied — a silent gap if anyone runs this without `--cost-per-minute` and reads \$0 as a real answer).

Every assumption baked in, and sensitivity actually measured this session:

Assumption	Default	Measured sensitivity
<code>calls_per_dau</code>	0.25	0.05→0.50 moves workers 37→362 and cost \$7K→\$70K/day — perfectly linear, and ungrounded : no external benchmark exists for what fraction of a hotel app's DAU calls the reservations line
<code>duration_minutes</code>	4.0	Not separately swept, but appears linearly in <code>daily_minutes</code> , hence in concurrency and cost identically to <code>calls_per_dau</code>
<code>peak_factor</code>	8.0	3→16 moves workers 68→362 — linear, but at least loosely groundable against real call-center traffic-shape data, unlike <code>calls_per_dau</code>
<code>sessions_per_worker</code>	40	5→80 moves workers 1445→91 (16x) — and this one conflicts with measured reality : this review's own data (§4) shows single turns blocking 5-24 seconds on synchronous, blocking TTS (<code>subprocess.run</code>), and the current reference implementation (<code>voice_loop.py</code>) has no concurrency model at all — "40 sessions per worker" presumes an async or thread-per-call architecture that does not exist in this codebase today
<code>headroom</code>	0.30	Additive 30% on top of peak; not separately swept
<code>cost_per_minute</code>	0.0	Silent \$0 unless explicitly overridden
<i>(implicit)</i> <code>daily_minutes / 1440</code>	—	Assumes perfectly uniform call volume across all 24 hours globally — no timezone clustering, no business-hours concentration, no day-of-week variation

Which assumption, if wrong, hurts most: `sessions_per_worker`, not because it's the least grounded (`calls_per_dau` is less grounded) but because it's the one assumption this review can directly contradict with measured data rather than merely note as "unverified." A hotel voice line's `calls_per_dau` is a genuine unknown that a real product would have real analytics for before this model even mattered. `sessions_per_worker=40`, by contrast, isn't unknown because nobody measured it — it's *wrong on its face* against this codebase's actual architecture, today, provably: a worker running `voice_loop.py`'s synchronous cascade cannot serve 40 concurrent sessions without an async rewrite, full stop, and every downstream number (workers, and therefore infrastructure cost) is quietly built on an assumption the code itself falsifies.

7. What must change before production

- **Room-native LiveKit agent worker** — `livekit/talk_server.py` is confirmed (from source, §6 of `SETUP-AND-REVIEW.md`) to be an HTTP request/response bridge, not a worker subscribing to and publishing LiveKit audio tracks. This is a real architecture gap, not a naming quibble — it means the "room" concept currently does no work beyond token issuance.
- **Session persistence + resumable state** — every session today is an in-memory Agent object (`_agent_sessions` in `talk_server.py`, or a single process in `voice_loop.py`); a worker restart or crash mid-call loses everything, silently.
- **Distributed cancellation (barge-in across workers)** — barge-in exists only in the browser demo's client-side heuristics; nothing coordinates an interruption across a real distributed worker fleet.
- **SIP/telephony dispatch** — `mocks/` demonstrates the *concept* mapping (REFER, BYE) with no real trunk, SBC, or dispatch rule anywhere in this repo.
- **Booking tool: real inventory, authn/authz, idempotency keys, audit log** — `run_tool()`'s `create_booking` returns the literal string "AH-4827" every single time, live model or mock, confirmed directly in this review's live testing (`stage2-evidence/02_booking_completion.log`). This is not a corner case; it is the default and only behavior. `booking.duplicate_confirmation` (added to the eval suite this review, `EVAL_REPORT.md` §7) makes this an explicit, executable fact rather than a claim.
- **Data: PII retention policy, recording consent, regional routing** — `telemetry.py`'s redaction is a good start but is not a retention policy; there is no consent flow anywhere in this system, and no data-residency logic at all.
- **Release safety: eval gate in CI, canary, rollback** — the 14-case suite (`EVAL_REPORT.md`) is a real, fast, deterministic gate against `agent.py`'s decision logic, but nothing in this repo runs it in CI, nothing gates a deploy on it, and there is no canary or rollback mechanism anywhere.

8. Verdict

Fit for a workshop or demo: yes, cleanly. The cascade architecture is genuinely well-taught by this codebase — clean separation of concerns, a real mock/live provider swap that actually works, solid guardrail and grounding behavior that held under adversarial testing, and a telemetry design whose redaction approach is good practice, not just workshop dressing.

Explicit list of what would block a pilot (real callers, even a small volume, even with human oversight):

1. The transfer/hangup compound-action bug (§2, §3) — a caller asking for a human has a measured, material chance of being disconnected instead. This alone should block any pilot until fixed.
2. Zero retry/fallback around any live provider call (§3) — any transient failure currently ends the call with no signal, not a graceful degradation.
3. No real booking persistence or idempotency (§7) — every "confirmed" booking in a pilot would be the same fake confirmation number, with no real reservation created anywhere.
4. No first-audio latency measurement or streaming TTS (§4) — the ~14-second measured time-to-first-response is not viable for a live phone call regardless of how good the conversation is once it starts.
5. `sessions_per_worker=40` is unvalidated against this codebase's actual (lack of) concurrency model (§6) — any capacity plan built on `scale_check.py`'s defaults today would be built on a number the code cannot currently deliver.

None of these are hard to state after this review, and none of them were visible from "the evals pass" alone — every one of them came from either manually operating the system (real mic, live provider, adversarial phrasing) or reading the actual control-flow code line by line. That is, in one sentence, the job this document was asked to do.